A Project Report On

Hostel Visitor Management System

Submitted by: OJASVINI KAPOOR

Enrollment number: 08814803124

Semester: 4th Sem B.Tech (4I5)

Course: Database Management System

Submitted To : Dr. Vasudha Bahl

Department of Information Technology

Maharaja Agrasen Institute of Technology

CONTENTS

1. INTRODUCTION
2. AIM
3. OBJECTIVES
4. FEATURES
5. SYSTEM OVERVIEW
6. ENTITY-RELATIONSHIP DIAGRAM
7. DATABASE SCHEMA
8. IMPLEMENTATION
9. OUTPUT
10. ADVANTAGES
11. CONCLUSION

1. INTRODUCTION

The Hostel Visitor Management System is a database-based solution designed to manage and monitor visitor entry in a hostel. In traditional systems, visitor records are maintained manually, which can lead to errors, data loss, and security issues.

This system provides a structured and digital approach where every visitor is recorded, approved, and logged. It ensures that only authorized visitors are allowed entry and all movements are tracked properly for security and audit purposes.

2. AIM

The aim of this project is to design and implement a database system that manages hostel visitor entries using an approval-based mechanism and maintains accurate records of all visitor activities.

3. OBJECTIVES

The main objectives of this project are:

- To maintain a proper record of all visitors entering the hostel.
- To implement an approval-based system for visitor entry .
- To ensure data consistency and avoid redundancy using normalization.
- To provide quick retrieval of visitor information.
- To improve overall hostel security through proper data management.

4. FEATURES

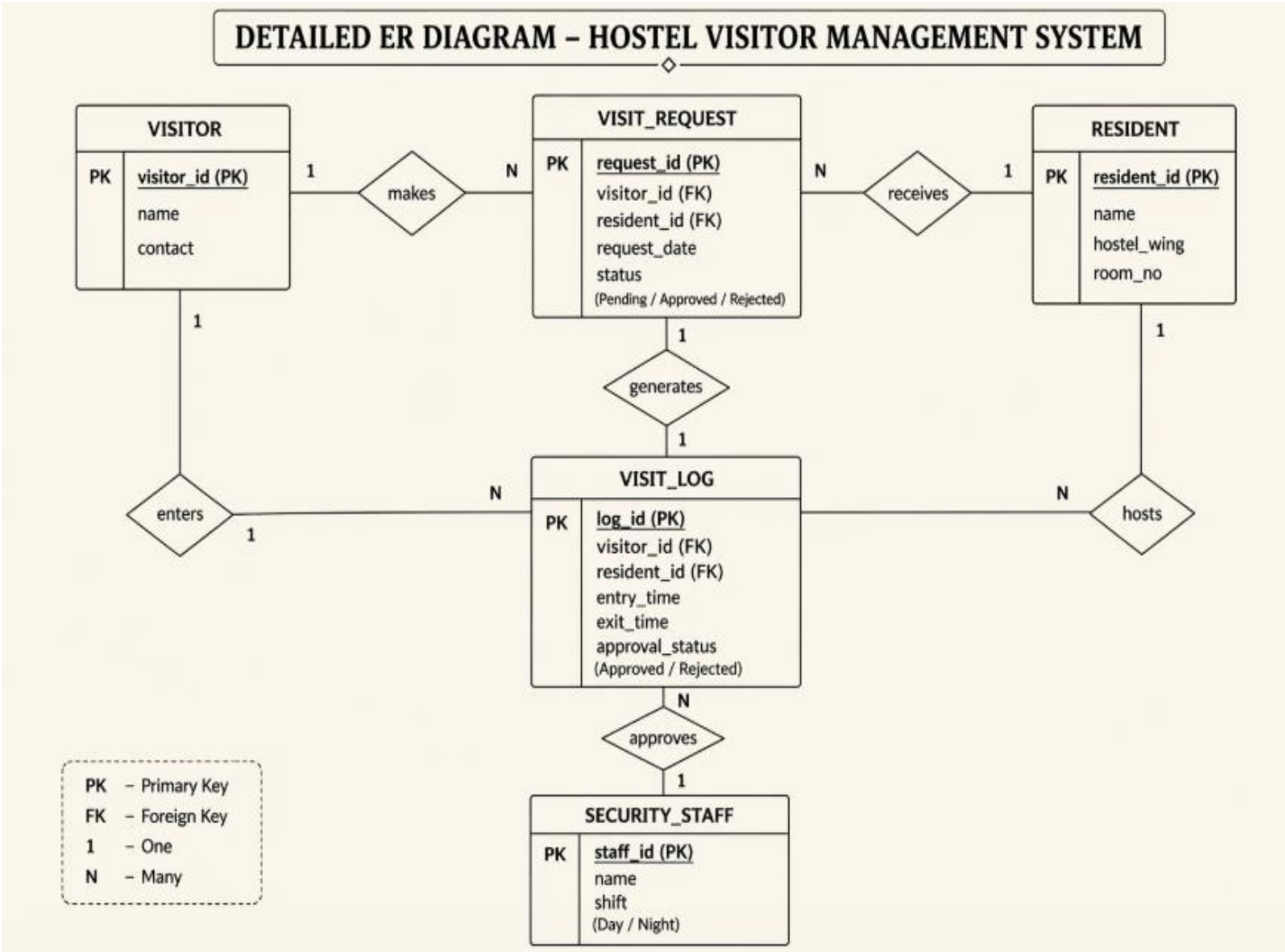
- Visitor registration and management
- Approval-based visit request system
- Real-time entry and exit logging
- Structured relational database design
- Efficient search and retrieval using SQL queries
- Secure data storage with relationships and constraints

5. SYSTEM OVERVIEW

The system operates in a step-by-step workflow. A visitor first submits a request to meet a resident. This request is reviewed by the resident or security staff and either approved or rejected.

Once approved, the visitor is allowed entry, and the details such as entry time, exit time, and approval status are stored in the VisitLog table. This ensures proper tracking and monitoring of all visitor activities.

6. ENTITY-RELATIONSHIP DIAGRAM



7. DATABASE SCHEMA

- **Visitor Table**

  - VisitorID (INT, PRIMARY KEY)
  - Name (VARCHAR(50), NOT NULL)
  - Contact (VARCHAR(15), UNIQUE)

- **Resident Table**

  - ResidentID (INT, PRIMARY KEY)
  - Name (VARCHAR(50), NOT NULL)
  - HostelWing (VARCHAR(10))
  - RoomNo (INT)

- **SecurityStaff Table**

  - StaffID (INT, PRIMARY KEY)
  - Name (VARCHAR(50), NOT NULL)
  - Shift (VARCHAR(20))

- **VisitRequest Table**

  - RequestID (INT, PRIMARY KEY)
  - VisitorID (INT, FOREIGN KEY REFERENCES Visitor(VisitorID))
  - ResidentID (INT, FOREIGN KEY REFERENCES Resident(ResidentID))
  - RequestDate (DATE)
  - Status (VARCHAR(20))

- **VisitLog Table**

  - LogID (INT, PRIMARY KEY)
  - VisitorID (INT, FOREIGN KEY REFERENCES Visitor(VisitorID))
  - ResidentID (INT, FOREIGN KEY REFERENCES Resident(ResidentID))
  - EntryTime (DATETIME)
  - ExitTime (DATETIME)
  - ApprovalStatus (VARCHAR(20))

8. IMPLEMENTATION

The database is implemented using SQL. Tables are created using CREATE TABLE statements with proper constraints such as PRIMARY KEY and FOREIGN KEY.

Sample data is inserted using INSERT statements to simulate real-world scenarios. The system supports operations like adding visitors, managing requests, and recording visit logs. The design ensures data integrity and follows normalization principles.

9. OUTPUT

- **Show All Tables**

```
MariaDB [HostelDB]> SHOW TABLES;
+--------------------+
| Tables_in_hosteldb |
+--------------------+
| resident           |
| securitystaff      |
| visitlog           |
| visitor            |
| visitrequest       |
+--------------------+
5 rows in set (0.001 sec)

MariaDB [HostelDB]> |
```

- **Show Structure**

```
MariaDB [HostelDB]> DESC Visitor;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| VisitorID | int(11)     | NO   | PRI | NULL    |       |
| Name      | varchar(50) | NO   |     | NULL    |       |
| Contact   | varchar(15) | YES  | UNI | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
3 rows in set (0.033 sec)

MariaDB [HostelDB]> DESC VisitRequest;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| RequestID   | int(11)     | NO   | PRI | NULL    |       |
| VisitorID   | int(11)     | YES  | MUL | NULL    |       |
| ResidentID  | int(11)     | YES  | MUL | NULL    |       |
| RequestDate | date        | YES  |     | NULL    |       |
| Status      | varchar(20) | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
5 rows in set (0.019 sec)

MariaDB [HostelDB]> |
```

➢ View data

```
MariaDB [HostelDB]> -- View data
MariaDB [HostelDB]> SELECT * FROM Visitor;
+-----------+--------------+------------+
| VisitorID | Name         | Contact    |
+-----------+--------------+------------+
|         1 | Ram Sharma   | 9876543210 |
|         2 | Priya Kapoor | 9811122233 |
|         3 | Aman Gupta   | 9898989898 |
+-----------+--------------+------------+
3 rows in set (0.002 sec)

MariaDB [HostelDB]> SELECT * FROM Resident;
+------------+-------------+------------+--------+
| ResidentID | Name        | HostelWing | RoomNo |
+------------+-------------+------------+--------+
|        101 | Amit Singh  | A          |    201 |
|        102 | Neha Jain   | B          |    305 |
|        103 | Rohit Mehta | C          |    110 |
+------------+-------------+------------+--------+
3 rows in set (0.001 sec)

MariaDB [HostelDB]>
MariaDB [HostelDB]> -- Pending requests
MariaDB [HostelDB]> SELECT * FROM VisitRequest WHERE Status='Pending';
+-----------+-----------+------------+-------------+---------+
| RequestID | VisitorID | ResidentID | RequestDate | Status  |
+-----------+-----------+------------+-------------+---------+
|      1001 |         1 |        101 | 2026-04-02  | Pending |
+-----------+-----------+------------+-------------+---------+
1 row in set (0.001 sec)
```

➢ Pending requests

```
MariaDB [HostelDB]> -- Pending requests
MariaDB [HostelDB]> SELECT * FROM VisitRequest WHERE Status='Pending';
+-----------+-----------+------------+-------------+---------+
| RequestID | VisitorID | ResidentID | RequestDate | Status  |
+-----------+-----------+------------+-------------+---------+
|      1001 |         1 |        101 | 2026-04-02  | Pending |
+-----------+-----------+------------+-------------+---------+
1 row in set (0.001 sec)
```

➢ Join query

```
MariaDB [HostelDB]> -- Join query (important)
MariaDB [HostelDB]> SELECT
    ->     V.Name AS VisitorName,
    ->     R.Name AS ResidentName,
    ->     L.EntryTime
    -> FROM VisitLog L
    -> JOIN Visitor V ON L.VisitorID = V.VisitorID
    -> JOIN Resident R ON L.ResidentID = R.ResidentID;
+--------------+--------------+---------------------+
| VisitorName  | ResidentName | EntryTime           |
+--------------+--------------+---------------------+
| Ram Sharma   | Amit Singh   | 2026-04-02 10:00:00 |
| Priya Kapoor | Neha Jain    | 2026-04-03 14:00:00 |
+--------------+--------------+---------------------+
2 rows in set (0.001 sec)
```

- **Count visits**

```
MariaDB [HostelDB]> -- Count visits
MariaDB [HostelDB]> SELECT ResidentID, COUNT(*) AS TotalVisits
    -> FROM VisitLog
    -> GROUP BY ResidentID;
+------------+-------------+
| ResidentID | TotalVisits |
+------------+-------------+
|        101 |           1 |
|        102 |           1 |
+------------+-------------+
2 rows in set (0.001 sec)
```

10. ADVANTAGES

- Improves hostel security
- Reduces manual errors

- Maintains structured records
- Enables quick data retrieval
- Supports monitoring and auditing

11. CONCLUSION

The Hostel Visitor Management System successfully demonstrates the application of database management concepts in a real-world scenario. It provides a structured and efficient way to manage visitor entries using an approval-based mechanism.

The system ensures accurate record-keeping, reduces manual errors, and enhances hostel security by maintaining proper logs of all visitor activities.

Overall, this project highlights how a well-designed database system can improve data management, ensure consistency, and provide a reliable solution for security-related operations.